# LAPORAN PERTEMUAN 5

## Penyimpanan Lokal

## (SharedPreferences dan SQLite)




**Dosen Pengampu:**

JEFRY SUNUPURWA ASRI, S.Kom., M.Kom.

**Disusun oleh:**

Rizqi Bagas Wicaksono (20240801187)

**Mata Kuliah:**

Pemrograman Mobile (CR002)

**PROGRAM STUDI TEKNIK INFORMATIKA**

**FAKULTAS ILMU KOMPUTER**

**UNIVERSITAS ESA UNGGUL**

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Data yang disimpan di memori, seperti variabel dan State, akan hilang begitu aplikasi ditutup. Agar data tetap tersedia ketika aplikasi dibuka kembali, aplikasi perlu menyimpannya di penyimpanan perangkat (persistence). Jenis penyimpanan yang dipilih bergantung pada sifat datanya: pengaturan kecil cocok disimpan sebagai pasangan key-value, sedangkan data terstruktur dan jumlahnya banyak, seperti catatan atau transaksi, lebih tepat disimpan dalam database.

Pada Pertemuan 5 ini, praktikum membahas dua jenis penyimpanan lokal pada Flutter, yaitu shared_preferences untuk pengaturan sederhana dan SQLite melalui paket sqflite untuk data terstruktur dengan operasi CRUD (Create, Read, Update, Delete). Sebagai penerapan, dibuat aplikasi Pencatat Pengeluaran.

## 1.2 Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memilih jenis penyimpanan lokal yang tepat: key-value, database, atau berkas.
2. Menyimpan dan membaca pengaturan sederhana dengan shared_preferences.
3. Merancang tabel dan membuat database SQLite dengan sqflite.
4. Mengimplementasikan operasi CRUD dan menghubungkannya dengan UI.
5. Memastikan data tetap ada setelah aplikasi ditutup dan dibuka kembali.

# BAB II
# LANDASAN TEORI

## 2.1 Pilihan Penyimpanan Lokal

Flutter menyediakan beberapa pilihan penyimpanan lokal. Perbandingannya ditunjukkan pada Tabel 1.

| Pilihan | Cocok untuk | Paket |
|---|---|---|
| Key-value | Pengaturan kecil: tema, nama, status login, angka terakhir | shared_preferences |
| Database SQLite | Data terstruktur dan banyak: catatan, transaksi, daftar produk | sqflite, path |
| Berkas | Dokumen, gambar, ekspor/impor data | path_provider, dart:io |

*Tabel 1. Pilihan penyimpanan lokal pada Flutter*

## 2.2 CRUD dan SQL

Setiap operasi CRUD memiliki padanan perintah SQL dan method pada sqflite, seperti pada Tabel 2.

| CRUD | SQL | Method sqflite |
|---|---|---|
| Create | INSERT | db.insert() |
| Read | SELECT | db.query() |
| Update | UPDATE | db.update() |
| Delete | DELETE | db.delete() |

*Tabel 2. Padanan operasi CRUD, SQL, dan method sqflite*

Query sebaiknya tidak disusun dengan menyambung teks dari pengguna, karena rawan SQL injection. Sebagai gantinya dipakai tanda tanya sebagai placeholder melalui where: 'id = ?' dan whereArgs: [id].

# BAB III
# HASIL PRAKTIKUM

## 3.1 Alat dan Bahan

- Flutter SDK, editor, dan perangkat Android (mode debug). Paket sqflite paling stabil dijalankan di Android atau iOS.
- Proyek Flutter baru (praktikum_5) dengan paket shared_preferences, sqflite, dan path.
- Koneksi internet hanya untuk mengunduh paket; setelah itu tidak diperlukan.

## 3.2 Bagian A: SharedPreferences (Pengaturan Sederhana)

Paket dipasang dengan perintah flutter pub add shared_preferences. Aplikasi Pengaturan menyimpan tiga data: nama (String), pilihan mode gelap (bool), dan jumlah berapa kali aplikasi dibuka (int). Ketika halaman dibuat, method _muat() membaca ketiga data tersebut dari SharedPreferences. Nilai bawaan diberikan dengan operator ?? agar tidak bernilai null pada pembacaan pertama, lalu hitungan buka ditambah satu dan disimpan kembali. Tombol Simpan Nama menyimpan isi kolom nama dan menampilkan SnackBar "Nama disimpan", sedangkan Switch Mode Gelap langsung menyimpan pilihan tema saat diubah.

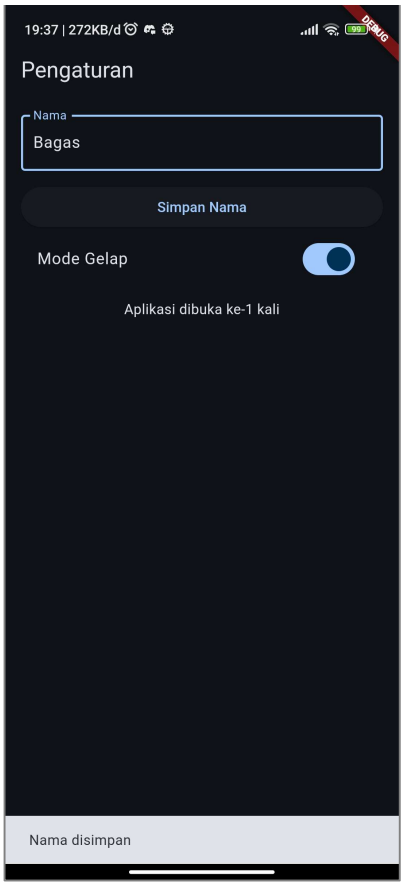
```
Future<void> _muat() async {
  final prefs = await SharedPreferences.getInstance();
  if (!mounted) return;
  setState(() {
    _controller.text = prefs.getString('nama') ?? '';
    _gelap = prefs.getBool('gelap') ?? false;
    _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
  });
  await prefs.setInt('bukaKe', _bukaKe);
}

Future<void> _ubahTema(bool nilai) async {
  setState(() => _gelap = nilai);
  final prefs = await SharedPreferences.getInstance();
  await prefs.setBool('gelap', nilai);
}
```

Pada Gambar 1, nama "Bagas" telah disimpan (muncul SnackBar "Nama disimpan"), mode gelap nonaktif, dan teks di bawahnya menunjukkan aplikasi dibuka ke-1 kali. Pada Gambar 2, Switch Mode Gelap diaktifkan sehingga seluruh tampilan berganti ke tema gelap.




*Gambar 1. Halaman Pengaturan (mode terang)*




*Gambar 2. Halaman Pengaturan (mode gelap)*

### 3.3 Bagian B: Menyiapkan SQLite

Paket dipasang dengan flutter pub add sqflite path. Dibuat model Catatan (id, judul, isi) dengan method toMap() dan factory fromMap() untuk menerjemahkan objek ke baris tabel dan sebaliknya. Kelas pembantu DbHelper membuka database catatan.db, membuat tabel catatan pada onCreate, dan menyediakan method tambah, semua, ubah, dan hapus. Kolom id bernilai null saat catatan baru dibuat, lalu diisi otomatis oleh SQLite karena memakai AUTOINCREMENT.

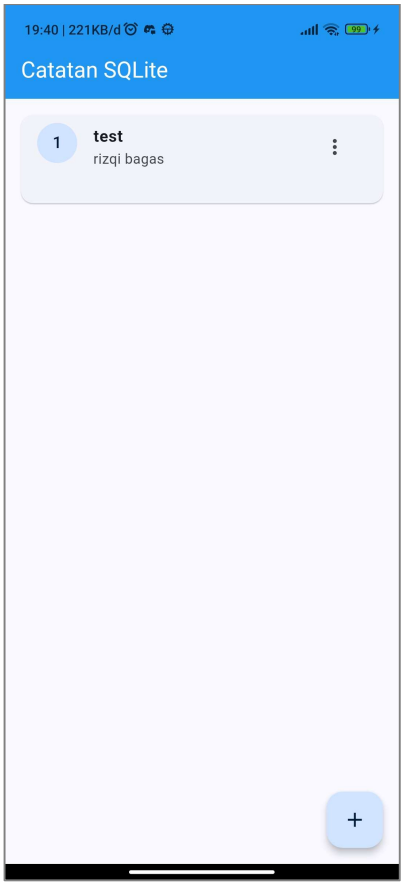
```
onCreate: (db, version) {
  return db.execute(
    'CREATE TABLE catatan('
    'id INTEGER PRIMARY KEY AUTOINCREMENT, '
    'judul TEXT NOT NULL, '
    'isi TEXT NOT NULL)',
  );
},

static Future<int> ubah(Catatan c) async {
  final db = await database;
  return db.update('catatan', c.toMap(), where: 'id = ?', whereArgs: [c.id]);
}
```

Pada percobaan, aplikasi bernama Catatan SQLite menampilkan tombol tambah (+). Ketika ditekan, muncul dialog Tambah Catatan berisi kolom Judul, Isi Catatan, serta tombol Batal dan Simpan (Gambar 4). Setelah catatan dengan judul "test" dan isi "rizqi bagas" disimpan, catatan tersebut tampil pada daftar sebagai kartu dengan nomor 1 dan menu titik tiga di sisi kanan (Gambar 5).




*Gambar 4. Dialog Tambah Catatan*




*Gambar 5. Daftar catatan setelah data disimpan*

### 3.4 Bagian C: Halaman Daftar Catatan (Read dan Delete)

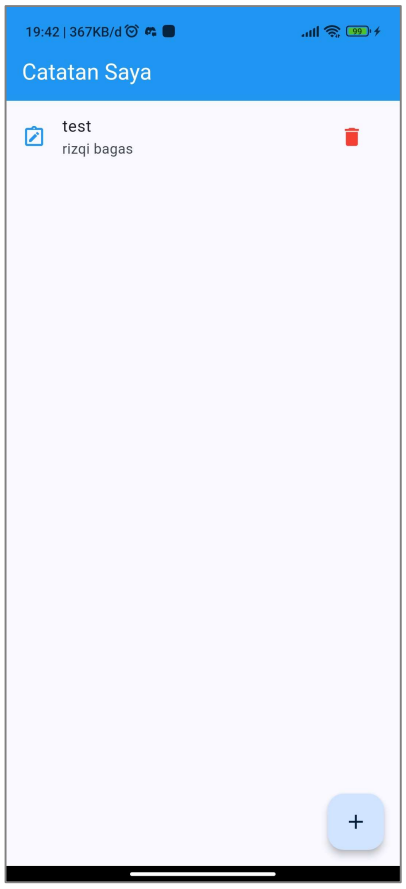
Halaman CatatanPage membaca seluruh catatan dengan DbHelper.semua() melalui FutureBuilder, dengan tiga keadaan: loading, galat, dan data (disertai teks "Belum ada catatan" bila kosong). Setiap baris berupa ListTile dengan judul, isi (maksimal dua baris), dan ikon sampah merah untuk menghapus catatan. Setelah catatan dihapus atau setelah kembali dari halaman form, method _muat() dipanggil agar Future dibuat ulang dan daftar diperbarui.

Gambar 6 menunjukkan halaman Catatan Saya yang memuat catatan "test" yang sama

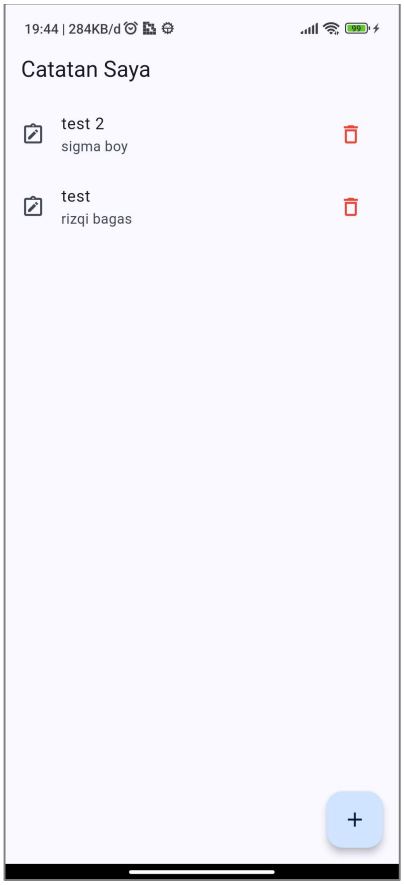
dengan yang disimpan pada Bagian B, padahal aplikasi telah dijalankan ulang dengan tampilan baru. Hal ini menandakan bahwa data tersimpan di database dan tidak hilang ketika aplikasi ditutup.

### 3.5 Bagian D: Form Tambah dan Ubah Catatan (Create dan Update)

Satu halaman FormCatatanPage dipakai untuk dua keperluan. Jika parameter catatan kosong berarti tambah (memanggil DbHelper.tambah), dan jika terisi berarti ubah (memanggil DbHelper.ubah). Kedua kolom memiliki validasi sehingga judul dan isi wajib diisi. Setelah menambah satu catatan lagi, daftar pada Gambar 7 memuat dua catatan: "test 2" dan "test", dengan catatan terbaru berada di paling atas karena query memakai orderBy: 'id DESC'.




*Gambar 6. Halaman Catatan Saya (satu catatan)*




*Gambar 7. Halaman Catatan Saya setelah menambah catatan*

### 3.6 Tugas: Aplikasi Pencatat Pengeluaran

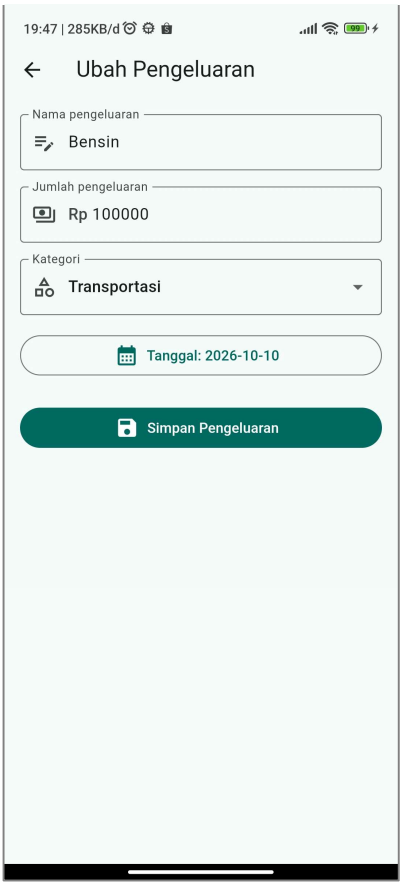
Tugas pertemuan ini adalah membuat aplikasi Pencatat Pengeluaran. Rancangan aplikasi

mengikuti ketentuan tugas dan dirangkum pada Tabel 3.

| Ketentuan | Penerapan |
|---|---|
| Tabel pengeluaran | Tabel pengeluaran dengan kolom id (INTEGER PRIMARY KEY AUTOINCREMENT), nama (TEXT), jumlah (INTEGER), kategori (TEXT), dan tanggal (TEXT, format yyyy-MM-dd) |
| CRUD lengkap | DbHelper menyediakan tambah (insert), semua (query), ubah (update), dan hapus (delete); daftar memuat ulang data setelah setiap perubahan |
| Validasi form | Nama wajib diisi; jumlah harus berupa angka lebih dari 0; kategori dipilih dari dropdown |
| Total pengeluaran | Ditampilkan di halaman utama, dihitung dengan SELECT SUM(jumlah) |
| Pengaturan | Satu pengaturan (mode gelap) disimpan dengan shared_preferences |
| Data bertahan | Data tersimpan di SQLite sehingga tetap ada setelah aplikasi ditutup dan dibuka kembali |

*Tabel 3. Rancangan aplikasi Pencatat Pengeluaran*

Gambar 8 menampilkan halaman Ubah Pengeluaran yang dibuka dari daftar untuk mengubah data yang sudah ada. Form berisi kolom Nama pengeluaran (terisi "Bensin"), kolom Jumlah pengeluaran dengan awalan "Rp" (terisi 100000), dropdown Kategori (terpilih "Transportasi"), tombol pemilih tanggal (Tanggal: 2026-10-10), dan tombol Simpan Pengeluaran. Judul halaman berubah menjadi "Ubah Pengeluaran" karena form yang sama dipakai untuk tambah dan ubah, mengikuti pola pada Bagian D.




*Gambar 8. Form Ubah Pengeluaran*

# BAB IV
# PEMBAHASAN

Berikut jawaban atas pertanyaan refleksi pada modul praktikum.

## 4.1 Mengapa shared_preferences kurang cocok untuk menyimpan ratusan catatan? Apa pilihan yang lebih baik?

shared_preferences dirancang untuk pasangan key-value kecil seperti nama, tema, atau status login. Seluruh datanya dibaca ke memori sekaligus, tidak ada struktur tabel, dan tidak ada kemampuan mencari, menyaring, mengurutkan, atau mengubah satu data tanpa membaca dan menulis ulang keseluruhannya. Bila ratusan catatan dijejalkan ke dalamnya (misalnya sebagai satu string JSON), pembacaan menjadi lambat, boros memori, dan sulit dikelola. Pilihan yang lebih baik adalah database SQLite melalui sqflite, karena mendukung tabel terstruktur, query SQL (WHERE, ORDER BY, SUM), serta operasi CRUD per baris.

## 4.2 Mengapa query memakai placeholder ? dan whereArgs, bukan menyambung teks langsung?

Jika query disusun dengan menyambung teks masukan pengguna, pengguna berniat jahat dapat menyisipkan potongan perintah SQL melalui masukannya sehingga struktur query berubah (SQL injection), misalnya untuk membaca atau menghapus data yang bukan haknya. Dengan placeholder ?, perintah SQL dan nilai dikirim terpisah; nilai pada whereArgs selalu diperlakukan sebagai data, bukan sebagai bagian perintah. Cara ini juga menghindari kesalahan akibat karakter khusus seperti tanda kutip pada teks.

## 4.3 Mengapa setelah kembali dari halaman form, daftar perlu dimuat ulang? Bagaimana caranya pada kode ini?

FutureBuilder hanya mengamati Future yang diberikan kepadanya, dan Future tersebut dibuat sekali di initState(). Perubahan di database yang terjadi pada halaman lain tidak otomatis diketahui oleh daftar, sehingga tanpa dimuat ulang daftar akan tetap menampilkan data lama. Pada kode ini, method _buka() menunggu hasil Navigator.push dengan await. Setelah halaman form ditutup, eksekusi dilanjutkan dan _muat() dipanggil, yang membuat Future baru dari DbHelper.semua() di dalam setState() sehingga FutureBuilder membangun ulang daftar dengan

data terbaru. Pemeriksaan if (!mounted) return; dilakukan terlebih dahulu agar setState tidak dipanggil pada widget yang sudah tidak aktif.

**4.4 Apa fungsi onCreate dan onUpgrade, dan kapan masing-masing dijalankan?**

onCreate dijalankan satu kali, yaitu ketika file database belum ada dan baru dibuat (misalnya saat aplikasi pertama kali dijalankan). Fungsinya membuat tabel awal. onUpgrade dijalankan ketika database sudah ada di perangkat tetapi nomor version yang diberikan pada openDatabase lebih tinggi daripada versi database yang tersimpan. Fungsinya memigrasikan skema tanpa menghapus data, misalnya dengan ALTER TABLE untuk menambah kolom. Bila skema diubah tanpa menaikkan version, onCreate tidak dijalankan lagi sehingga muncul galat seperti no such table atau no such column.

# BAB V
# KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Data di memori hilang saat aplikasi ditutup, sehingga data yang perlu bertahan harus disimpan di penyimpanan perangkat. Jenis penyimpanan dipilih sesuai kebutuhan: key-value untuk pengaturan kecil dan database untuk data terstruktur.
2. shared_preferences mudah dipakai untuk menyimpan pengaturan sederhana seperti nama, mode gelap, dan hitungan buka aplikasi, dengan nilai bawaan (??) untuk menghindari null.
3. sqflite memungkinkan pembuatan tabel dan operasi CRUD melalui DbHelper dan model dengan toMap/fromMap, dengan query yang aman dari SQL injection berkat placeholder dan whereArgs.
4. Daftar yang memakai FutureBuilder harus dimuat ulang (membuat Future baru) setelah ada perubahan data agar tampilan sesuai dengan isi database.
5. Aplikasi Pencatat Pengeluaran menerapkan CRUD, validasi form, total pengeluaran, serta penyimpanan pengaturan, dengan data yang tetap tersimpan setelah aplikasi ditutup.

# LAMPIRAN

# KODE PROGRAM TUGAS PENCATAT PENGELUARAN

## Lampiran 1. Model Pengeluaran

```
class Pengeluaran {
  final int? id;
  final String nama;
  final int jumlah;
  final String kategori;
  final String tanggal; // format yyyy-MM-dd

  const Pengeluaran({
    this.id,
    required this.nama,
    required this.jumlah,
    required this.kategori,
    required this.tanggal,
  });

  Map<String, Object?> toMap() => {
        'id': id,
        'nama': nama,
        'jumlah': jumlah,
        'kategori': kategori,
        'tanggal': tanggal,
      };

  factory Pengeluaran.fromMap(Map<String, Object?> m) => Pengeluaran(
        id: m['id'] as int,
        nama: m['nama'] as String,
        jumlah: m['jumlah'] as int,
        kategori: m['kategori'] as String,
        tanggal: m['tanggal'] as String,
      );
}
```

## Lampiran 2. DbHelper

```
class DbHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'pengeluaran.db');
    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE pengeluaran('
          'id INTEGER PRIMARY KEY AUTOINCREMENT, '
          'nama TEXT NOT NULL, '
          'jumlah INTEGER NOT NULL, '
          'kategori TEXT NOT NULL, '
          'tanggal TEXT NOT NULL)',
        );
      },
    );
    return _db!;
```

```
  }

  static Future<int> tambah(Pengeluaran x) async {
    final db = await database;
    return db.insert('pengeluaran', x.toMap());
  }

  static Future<List<Pengeluaran>> semua() async {
    final db = await database;
    final rows = await db.query('pengeluaran',
        orderBy: 'tanggal DESC, id DESC');
    return rows.map(Pengeluaran.fromMap).toList();
  }

  static Future<int> ubah(Pengeluaran x) async {
    final db = await database;
    return db.update('pengeluaran', x.toMap(),
        where: 'id = ?', whereArgs: [x.id]);
  }

  static Future<int> hapus(int id) async {
    final db = await database;
    return db.delete('pengeluaran', where: 'id = ?', whereArgs: [id]);
  }

  static Future<int> total() async {
    final db = await database;
    final r = await db.rawQuery(
        'SELECT COALESCE(SUM(jumlah), 0) FROM pengeluaran');
    return Sqflite.firstIntValue(r) ?? 0;
  }
}
```

## Lampiran 3. Form Tambah dan Ubah Pengeluaran

```
const kategoriList = [
  'Makanan', 'Transportasi', 'Belanja', 'Tagihan', 'Lainnya',
];

class FormPengeluaranPage extends StatefulWidget {
  final Pengeluaran? data;
  const FormPengeluaranPage({super.key, this.data});
  @override
  State<FormPengeluaranPage> createState() => _FormPengeluaranPageState();
}

class _FormPengeluaranPageState extends State<FormPengeluaranPage> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _nama;
  late final TextEditingController _jumlah;
  String? _kategori;
  late DateTime _tanggal;

  @override
  void initState() {
    super.initState();
    final d = widget.data;
    _nama = TextEditingController(text: d?.nama ?? '');
    _jumlah = TextEditingController(text: d == null ? '' : '${d.jumlah}');
    _kategori = d?.kategori;
    _tanggal = d == null ? DateTime.now() : DateTime.parse(d.tanggal);
  }
```

```
@override
void dispose() {
  _nama.dispose();
  _jumlah.dispose();
  super.dispose();
}

String get _tanggalTeks => _tanggal.toIso8601String().substring(0, 10);

Future<void> _pilihTanggal() async {
  final hasil = await showDatePicker(
    context: context,
    initialDate: _tanggal,
    firstDate: DateTime(2000),
    lastDate: DateTime(2100),
  );
  if (hasil != null) setState(() => _tanggal = hasil);
}

Future<void> _simpan() async {
  if (!_formKey.currentState!.validate()) return;
  final x = Pengeluaran(
    id: widget.data?.id,
    nama: _nama.text.trim(),
    jumlah: int.parse(_jumlah.text.trim()),
    kategori: _kategori!,
    tanggal: _tanggalTeks,
  );
  if (widget.data == null) {
    await DbHelper.tambah(x);
  } else {
    await DbHelper.ubah(x);
  }
  if (!mounted) return;
  Navigator.pop(context);
}

@override
Widget build(BuildContext context) {
  final baru = widget.data == null;
  return Scaffold(
    appBar: AppBar(
        title: Text(baru ? 'Tambah Pengeluaran' : 'Ubah Pengeluaran')),
    body: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _nama,
            decoration: const InputDecoration(
              labelText: 'Nama pengeluaran',
              prefixIcon: Icon(Icons.edit_note),
              border: OutlineInputBorder(),
            ),
            validator: (v) => (v == null || v.trim().isEmpty)
                ? 'Nama wajib diisi'
                : null,
          ),
          const SizedBox(height: 12),
          TextFormField(
            controller: _jumlah,
            keyboardType: TextInputType.number,
            decoration: const InputDecoration(
```

```
                labelText: 'Jumlah pengeluaran',
                prefixIcon: Icon(Icons.payments_outlined),
                prefixText: 'Rp ',
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                final n = int.tryParse((v ?? '').trim());
                if (n == null) return 'Jumlah harus berupa angka';
                if (n <= 0) return 'Jumlah harus lebih dari 0';
                return null;
              },
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              value: _kategori,
              decoration: const InputDecoration(
                labelText: 'Kategori',
                prefixIcon: Icon(Icons.category_outlined),
                border: OutlineInputBorder(),
              ),
              items: kategoriList
                  .map((k) => DropdownMenuItem(value: k, child: Text(k)))
                  .toList(),
              onChanged: (v) => setState(() => _kategori = v),
              validator: (v) => v == null ? 'Pilih kategori' : null,
            ),
            const SizedBox(height: 12),
            OutlinedButton.icon(
              onPressed: _pilihTanggal,
              icon: const Icon(Icons.calendar_month),
              label: Text('Tanggal: $_tanggalTeks'),
            ),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: _simpan,
              icon: const Icon(Icons.save),
              label: const Text('Simpan Pengeluaran'),
            ),
          ],
        ),
      ),
    );
  }
}
```

## Lampiran 4. Halaman Daftar dan Total Pengeluaran

```
class PengeluaranPage extends StatefulWidget {
  final bool gelap;
  final ValueChanged<bool> onGantiTema;
  const PengeluaranPage(
      {super.key, required this.gelap, required this.onGantiTema});
  @override
  State<PengeluaranPage> createState() => _PengeluaranPageState();
}

class _PengeluaranPageState extends State<PengeluaranPage> {
  late Future<List<Pengeluaran>> _future;
  late Future<int> _total;

  @override
  void initState() {
    super.initState();
    _muat();
```

```
}

void _muat() {
  setState(() {
    _future = DbHelper.semua();
    _total = DbHelper.total();
  });
}

Future<void> _buka([Pengeluaran? x]) async {
  await Navigator.push(
    context,
    MaterialPageRoute(builder: (_) => FormPengeluaranPage(data: x)),
  );
  if (!mounted) return;
  _muat();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Pencatat Pengeluaran'),
      actions: [
        Switch(value: widget.gelap, onChanged: widget.onGantiTema),
      ],
    ),
    body: Column(
      children: [
        FutureBuilder<int>(
          future: _total,
          builder: (context, s) => Card(
            margin: const EdgeInsets.all(16),
            child: ListTile(
              title: const Text('Total pengeluaran'),
              trailing: Text('Rp ${s.data ?? 0}',
                  style: const TextStyle(
                      fontSize: 18, fontWeight: FontWeight.bold)),
            ),
          ),
        ),
        Expanded(
          child: FutureBuilder<List<Pengeluaran>>(
            future: _future,
            builder: (context, snapshot) {
              if (snapshot.connectionState == ConnectionState.waiting) {
                return const Center(child: CircularProgressIndicator());
              }
              if (snapshot.hasError) {
                return Center(child: Text('Galat: ${snapshot.error}'));
              }
              final data = snapshot.data!;
              if (data.isEmpty) {
                return const Center(child: Text('Belum ada pengeluaran'));
              }
              return ListView.builder(
                itemCount: data.length,
                itemBuilder: (context, i) {
                  final x = data[i];
                  return ListTile(
                    title: Text(x.nama),
                    subtitle: Text('${x.kategori} • ${x.tanggal}'),
                    onTap: () => _buka(x),
                    trailing: Row(
                      mainAxisSize: MainAxisSize.min,
                      children: [
```

```
                          Text('Rp ${x.jumlah}'),
                          IconButton(
                            icon: const Icon(Icons.delete, color: Colors.red),
                            onPressed: () async {
                              await DbHelper.hapus(x.id!);
                              _muat();
                            },
                          ),
                        ],
                      ),
                    );
                  },
                );
              },
            ),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => _buka(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

## Lampiran 5. main.dart dan Pengaturan Mode Gelap

```
void main() => runApp(const MyApp());

class MyApp extends StatefulWidget {
  const MyApp({super.key});
  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  bool _gelap = false;

  @override
  void initState() {
    super.initState();
    _muatTema();
  }

  Future<void> _muatTema() async {
    final prefs = await SharedPreferences.getInstance();
    if (!mounted) return;
    setState(() => _gelap = prefs.getBool('gelap') ?? false);
  }

  Future<void> _gantiTema(bool nilai) async {
    setState(() => _gelap = nilai);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('gelap', nilai);
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Pencatat Pengeluaran',
      theme: ThemeData(
        brightness: _gelap ? Brightness.dark : Brightness.light,
```

```
        colorSchemeSeed: Colors.teal,
        useMaterial3: true,
      ),
      home: PengeluaranPage(gelap: _gelap, onGantiTema: _gantiTema),
    );
  }
}
```